# Instructor Handout 8: Order Is a Superpower

Searching & the Elementary Sorts — Week 8

Dr. Auqib Hamid Lone

*Department of Computer Science & Engineering, GCET Ganderbal*

October 16, 2026

**Instructor Copy — Not for Student Distribution**

Concepts That Click

## 8.1 Before You Teach This Lecture

- **Karumanchi, *Data Structures And Algorithms Made Easy*, Ch. 11, pp.547–589 (PDF)** — the search operation, linear search, binary search, and the lower/upper-bound variants; **Ch. 10, pp.504–546 (PDF)** — insertion, selection, bubble, and counting sort, plus the comparison-of-sorts table; **Ch. 15, pp.640–684 (PDF)** — string search, the anchor for the longest-common-prefix application. *Page-cited* (this edition has no printed folios — cite as "p. N (PDF)"); index built 2026-08-11. Language-neutral C-style pseudocode, so syntax transfers directly to CS301's C.

- **Wirth, *Algorithms and Data Structures*, §1.9.1, p.34** — linear search; **§1.9.2, pp.34–36** — binary search; **§2.2.1, p.46** — straight insertion; **§2.2.2, p.48** — straight selection; **§2.2.3, p.50** — straight exchange (the deck's bubble sort). *Page-cited*; index built 2026-08-11. All code is Oberon — cite for concepts and analysis, never for syntax; translate to C per the notation registry.

- **Sedgewick & Wayne, *Algorithms*, §1.4, p.172** — the "how long will my program take?" cost model reused from Weeks 1–2; **p.187, the order-of-growth table** — binary search is the canonical *logarithmic* row; **§2.1, p.244** — elementary sorts. *Page-cited.* Java code — cite for definitions and analysis only.

- **Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 7** (sorting) and **Knuth, *TAOCP* Vol. 3** (sorting and searching) — *not page-indexed*; cite at chapter level only. Note the KB's explicit warning: Knuth's sorting/searching material is in *Vol. 3*, not Vol. 1 (the syllabus lists Vol. 1 only) — always name Vol. 3 here.

- **Aho–Hopcroft–Ullman, *Data Structures and Algorithms*, Ch. 8** and **Neapolitan, *Foundations of Algorithms*, Ch. 2** — *not page-indexed*; chapter-level attribution only. Neapolitan is the strongest listed book on complexity analysis; use it for depth on the recurrences, not for a page.

- **Handoff from Week 7.** Week 7 closed the chain family (doubly linked lists, circular rings, list-backed stack/queue, polynomials) and promised "Searching — linear versus binary, and

the elementary sorts that put data in the order binary search needs." This week pays that off. It does *not* re-teach list surgery, and it reuses Week 2's $O/\Theta/\Omega$ and best/worst/average toolkit rather than re-deriving it. If students look lost at the opening, run the Week 7 summary frame first.

- **Deck corrections to trust.** Two items in this deck were fixed recently and the handout reflects the corrected versions: (1) the integer-square-root code casts `(long long) mid * mid` *before* the multiply — the widening is the point, not a stylistic flourish; (2) the "Check Your Prediction: the unsorted array" frame now searches the key 3 (for which `bsearch` genuinely returns $-1$), not the key 7 (which the old frame mis-stated — `bsearch` would have returned index 0).

- **Plan the board examples in advance.** Every section below carries two **Board:** examples plus a live-interaction hook. Rehearse silently the counting-sort scatter (the prefix-sum positions are easy to fumble live) and the two binary-search overflow instances — the numbers on the board must be right the first time.

## 8.2 Motivation and the Handoff

This Act is motivational and orienting — nothing in it is examined directly — but it sets the frame the technical sections lean on. Every lookup in the course so far has been a *walk*: the calculator's symbol table, a linked list since Week 6, answers a lookup by scanning from `head` until it matches or hits `NULL`, $O(n)$ worst case. Week 8's whole job is to show that *order* turns that $O(n)$ scan into an $O(\log n)$ probe — then to learn the cheapest ways to create the order.

**Open with the four questions, on the board, before any answer.** Write them in order and leave them up all lecture: (1) unordered or sorted — how much does order actually buy a search; (2) what precondition does binary search silently assume, and what happens when it is violated; (3) can we search for a value we never stored — the integer square root of $n$; (4) insertion, selection, bubble, counting — which sort does each input reward, and at what cost. Compress the plan into one line beneath them, which is also the order of every section: *write the search contract; price linear versus binary; search a value that is not in any array; then sort — four algorithms, their traces, and their honest bills.*

**Board: three labelled boxes for the plan.** Draw three boxes labelled "price it," "search the un-stored," "sort it" (questions 1–2 collapse into the first). Invite one-word guesses from the class before revealing anything, and leave them unfilled for the Summary. This costs two minutes and gives the lecture a spine students can feel closing.

**The running thread: the calculator looks up an identifier.** The tokenizer has just produced the identifier `x`; the symbol table must answer *is `x` declared, and where?* As a linked list each lookup scans from `head` — up to 2,000 comparisons per token for a 2,000-identifier program. Keep the same entries *sorted by name*, and a binary search answers in about $\log_2 2{,}000 \approx 11$ comparisons — roughly 180× fewer.

**Board: the lookup arithmetic (distinct instance 1).** Write "2,000 identifiers" and, beneath it, the two costs side by side: a linear walk $\approx$ 2,000 comparisons/token; a binary probe $\approx \log_2 2{,}000 \approx 11$; annotate the arrow between them "same correctness, $\sim$180× less work." Then ask the class to *tell you* the two numbers for a larger program instead of you writing them.

**Board: distinct instance 2 — a bigger table, and the live hook.** Pose it as a prediction *before* revealing: "a program with 1,000,000 identifiers — how many probes does a sorted binary search need, and how many does the list walk need?" Let guesses land, then write: $\log_2 1{,}000{,}000 \approx$

Concepts That Click

20 versus up to 1,000,000 — a factor of roughly 50,000. Land the general lesson, which is the sentence that motivates the four questions: *doubling the table adds just one comparison to the binary search, but doubles the walk.* Two questions then follow for the rest of the lecture: how much does sorted order speed up search, and how do we get data sorted without paying more than we save?

> **Teaching note:** The trap on this opening is treating it as a warm-up to skip. The "every lookup so far has been a walk" line is the *entire* motivation for the week, and it echoes Week 1's "the answer stays right; the arrangement of the data is the difference." Say the $\log_2$ and the 180× number out loud, twice — students remember one concrete speedup far better than the promise of "logarithmic time." Do not derive log here; the recurrence derivation comes in its proper place in Section 8.3.

## 8.3 The Search Contract: Linear versus Binary

Finding a name in an *unordered* pile means checking one item at a time. Finding an entry in a *sorted* directory lets you jump to the middle and discard half at a glance. The difference is not cleverness — it is a precondition, and the precondition is the whole story of this Act.

**Definition (The search operation (Karumanchi Ch. 11, pp.547–589 (PDF); Wirth §1.9.1, p.34)).** *`search(A, n, key)` returns an index `i` with `A[i] == key`, or −1 if `key` does not occur.* ***Linear search*** *makes no assumption about the array.* ***Binary search*** requires *the array to be sorted in nondecreasing order — this precondition is part of the contract, not an optional extra.*

**Board: the contract, then the two arrays (instance 1).** Write the signature `search(A, n, key) -> i or -1` first, alone. Only then write the unordered `[7, 3, 9, 1, 5, 8]` and the sorted `[1, 3, 5, 7, 8, 9]` side by side, and search for the value 9 in each: in the first, scan left to right (7 no, 3 no, 9 hit at index 2); in the second, probe the middle (5), see $9 > 5$, keep `[7, 8, 9]`, probe its middle (8), keep `[9]`, found. Build order: contract, then arrays, then the two searches — never the searches before the precondition is named. **Live question:** "the values in the two arrays are the same — what is the only difference that admits the shortcut of the second search?" (The order.)

### 8.3.1 Linear search: scan until you find it

Linear search tests every element at most once, touching `A[0]`, then `A[1]`, and so on in index order — no assumption about the values is ever used, which is exactly why it works on any input. The deck's code:

```
int lsearch(int A[], int n, int key) {
  for (int i = 0; i < n; i++)
    if (A[i] == key) return i;   // hit: return the index
  return -1;                     // scanned the whole array: miss
}
```

| case | comparisons | example input |
|---|---|---|
| best | 1 | the first element matches |
| worst | $n$ | absent, or the last element |
| average (uniform random hit) | $(n+1)/2$ | the key is equally likely anywhere |

Concepts That Click

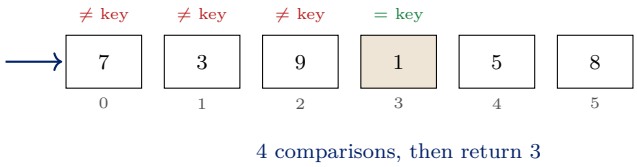



Figure 8.1: Linear search of `[7, 3, 9, 1, 5, 8]` for `key = 1`: three misses, then the hit at index 3 — four comparisons (cf. Karumanchi, Ch. 11, pp.547–589 (PDF)).

All three are $\Theta(n)$: linear search is a linear scan, and no ordering can change that. It is the honest baseline — and the price we pay for not owning sorted data.

**Board: walk the diagram (instance 1).** Draw the six empty cells first and label the indices 0..5 beneath them; write the values in a second pass, left to right. Then walk the search pointer cell by cell, writing the "≠ key" mark above each miss and only the "= key" mark above the hit — narrate every comparison out loud ("$7 \neq 1$; $3 \neq 1$; $9 \neq 1$; $1 = 1$ — return 3"). Build order: cells, indices, values, then the walk with its marks. The point to underline: nothing in the array told us that 1 could not have been anywhere ahead, so *every* earlier cell had to be checked.

**Board: distinct instance 2 — a worst-case-and-miss pair.** On the same six-cell array, send two students to the board: one searches for the key 8, the other for a key that is *absent*, say 4. The first walks $7, 3, 9, 1, 5, 8$ — six comparisons, return index 5. The second walks all six — six comparisons, return $-1$. Ask the class, before either writes: *"which of the two costs more?"* (Neither — both are the worst case, $n$ comparisons; a miss and a hit-at-the-end are the same cost. This is the point of the worst-case column, and it prepares the "binary search is better *once you own sorted data*" payoff.)

> **Teaching note:** Students routinely say linear search is "the slow one" and want to move on. The honest framing is that linear search is not slow — it is *unconditioned*: it is the best you can do when you refuse to pay for order, and for a short array, or an array that is searched once, it beats sorting-then-searching outright. Keep the judgment conditional: the question is never "which is faster" but "do you already own sorted data." This is the same discipline Week 1 taught for structures, applied to algorithms.

### 8.3.2 The sorted precondition — and what its absence costs

Binary search compares the target with the *middle* element, then keeps only the half that could contain it. That logic is sound *only if* everything left of the middle is smaller than it and everything right is larger — exactly the sorted invariant.

**Definition (Binary-search precondition (Karumanchi Ch. 11, pp.547–589 (PDF))).** *Binary search on $A[0..n-1]$ requires $A[0] \leq A[1] \leq \cdots \leq A[n-1]$. Applied to unsorted data it does not error — it returns a* plausible wrong answer*, silently discarding the half that really held the key. The precondition is a* contract*, not a hint: binary search is only as correct as the order it is handed.*

**Board: the failure trace with key 1 (instance 1).** Write `lo = 0, hi = 5`. Iteration 1: `mid = 0 + (5-0)/2 = 2`, `A[2] = 9`; since $1 < 9$, set `hi = 1` — and cross out indices 3..5 on the board, explicitly crossing through the 1 at index 3. Iteration 2: `lo = 0, hi = 1, mid = 0`, `A[0] = 7 > 1`, so `hi = -1`; now `lo > hi`, the range is empty, return $-1$. Build order: write the two bounds, then each iteration's `mid` and the half discarded, crossing out the discarded cells as you

Concepts That Click

go. Land the punch: the search returns a miss even though 1 sits at index 3 — a wrong answer, not an error.

**Board: distinct instance 2 — the key-3 prediction.** This is the deck's corrected "Check Your Prediction" instance and it makes a good live hook. Pose the multiple choice *before* revealing anything: the array `[7, 3, 9, 1, 5, 8]` is unsorted; you run binary search for the key 3 — does it return (A) index 1, (B) a wrong index or $-1$, or (C) a runtime error? Let the room vote. Then trace: `mid = 2` (value 9); $3 < 9$, so it discards 3..5 and continues left; the range collapses to $\{0\}$, the value 7 there is $> 3$, and the search returns $-1$ — silently losing the 3 at index 1. Answer: **B**. **The live hook:** "why is it not C — why does nothing catch this?" (Because the code contains no check of whether the input is sorted; it simply trusts the contract, and a wrong "greater-than" comparison discards the half that held the key.)

> **Teaching note:** This pair of traces is the single most important board work of the Act: unsorted input does not crash, it produces a *plausible* answer, which is far worse than a crash. Students who only ever see binary search succeed on sorted data never build the reflex to check the precondition — and this is exactly the anti-pattern the course KB flags ("teaching binary search before establishing the sorted precondition as an invariant"). Do the two failure traces *before* showing the working code, so correctness is never taken for granted.

### 8.3.3 Binary search: halve the range

The deck's binary search, written in C (Wirth §1.9.2, pp.34–36; Karumanchi Ch. 11, pp.547–589 (PDF)):

```
int bsearch(int A[], int n, int key) {
  int lo = 0, hi = n - 1;              // current [lo, hi], inclusive
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;      // avoids lo+hi overflow
    if (A[mid] == key) return mid;     // hit
    else if (A[mid] < key) lo = mid + 1; // key is in the right half
    else hi = mid - 1;                 // key is in the left half
  }
  return -1;                           // range empty: miss
}
```

Each iteration either stops or halves the live range $[\texttt{lo}, \texttt{hi}]$. Writing `lo + (hi - lo)/2` rather than `(lo + hi)/2` is not cosmetic: for large indices the sum can overflow `int` — Week 1's caution about C arithmetic in disguise.

**Board: draw the shrinking range (instance 1).** Draw the eight index ticks along the top (a ruler), then row 1 as the full range 0..7 with the blue rule at the middle. Narrate each successive row as a redraw *beneath* the previous one, halving the gold band and moving the rule: row 2 is the left half 0..3 (because the key is smaller than the middle), row 3 the right half of that, row 4 the single found slot. Build order: ruler, then top-to-bottom one row per iteration, halving each time. The line to say at each step: "one comparison, and roughly half the candidates are gone." **Live hook:** after row 1, cover rows 2–4 and ask the class *which* half survives — "the key is bigger than the middle, so we keep …?" (the right half).

**Board: distinct instance 2 — a concrete sorted array trace.** The range diagram is abstract; back it with a real array. On sorted `[1, 3, 5, 7, 8, 9]`, trace `bsearch` for the key 8: `lo = 0, hi = 5`; iteration 1 `mid = 2`, `A[2] = 5 < 8` so `lo = 3`; iteration 2 `lo = 3, hi = 5, mid = 4`, `A[4] = 8 == key`, return 4. Two comparisons. Then ask a student to run the same

Concepts That Click

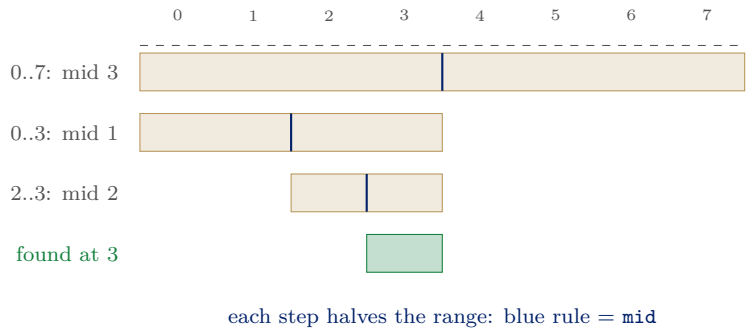



Figure 8.2: Binary search: each row halves the surviving range; the blue rule marks `mid` (cf. Wirth §1.9.2, pp.34–36).

array for the absent key 6: `mid = 2` ($5 < 6 \to$ `lo` $= 3$); `lo=3,hi=5,mid=4` ($8 > 6 \to$ `hi` $= 3$); `lo=3,hi=3,mid=3` ($7 > 6 \to$ `hi` $= 2$); `lo>hi`, return $-1$. Three comparisons for a miss. This is a genuinely different instance from the deck's eight-slot picture and exercises the exact code above.

**Board: the overflow trap, two instances.** Write the two candidate mid-computations side by side: `(lo + hi) / 2` and `lo + (hi - lo) / 2`. Instance 1: `lo = 1,500,000,000, hi = 2,000,000,000` — the sum 3,500,000,000 exceeds the 31-bit `int` maximum (2,147,483,647) and wraps negative; the safe form computes 1,500,000,000 + 250,000,000 with no overflow. Instance 2: `lo = 1,800,000,000, hi = 2,000,000,000` — sum wraps again, safe form gives 1,900,000,000. **Prediction hook:** pose instance 2 and have students say whether the *naive* form is safe before you reveal. Land the reuse: the same trap reappears as `mid * mid` in `isqrt` (Section 8.4) — it is Week 1's arithmetic caution, not a binary-search quirk.

### 8.3.4 Why binary search is $O(\log n)$

Let $T(n)$ be the worst-case number of comparisons for a range of size $n$. One comparison is spent, and a range of size at most $\lceil n/2 \rceil$ remains, so the cost obeys

$$T(n) = T(\lceil n/2 \rceil) + 1, \qquad T(1) = 1.$$

Unrolling the recurrence: after $k$ halvings the range has size $n/2^k$, which reaches 1 when $k = \log_2 n$. Hence $T(n) = \Theta(\log n)$ in the worst and average cases, and $O(1)$ in the best case (the target happens to be the middle element at once). The growth is unusually gentle:

| $n$ | 10 | 100 | 1,000 | 1,000,000 |
|---|---|---|---|---|
| comparisons $\approx \log_2 n$ | 4 | 7 | 10 | 20 |

Sedgewick classifies binary search as the canonical *logarithmic* order of growth (order-of-growth table, p.187; cost model, §1.4, p.172). Doubling $n$ adds just one comparison.

**Board: unroll the recurrence by hand.** Do not just state $\Theta(\log n)$ — write the halvings out: $n \to n/2 \to n/4 \to \cdots \to 1$, count the arrows ($k$), and set $n/2^k = 1$ so $k = \log_2 n$. Then fill the table row live: for $n = 1{,}000$, $\log_2 1000 \approx 10$; for $n = 1{,}000{,}000$, $\approx 20$. **Board: the doubling hook.** Ask the class to predict the comparison count at $n = 2{,}000{,}000$ before you write it — the whole point is that the answer is 21, not 40. Say the takeaway aloud: *doubling the input adds one comparison*, which is precisely the "gentle growth" the table shows.

Concepts That Click

### 8.3.5 Binary-search variants: first and last occurrence

The plain search returns *some* matching index. To find the *first* occurrence — useful for counting duplicates — keep the matching slot as a candidate instead of returning it, and continue into the left half (Karumanchi Ch. 11, pp.547–589 (PDF)):

```
int lowerBound(int A[], int n, int key) {   // first i with A[i] >= key
  int lo = 0, hi = n;                        // half-open [lo, hi)
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (A[mid] < key) lo = mid + 1;          // discard [lo, mid]
    else hi = mid;                           // keep mid as a candidate
  }
  return lo;                                 // n if every element < key
}
```

The occurrence count of `key` in a sorted array is then `upperBound` − `lowerBound`, computed in $O(\log n)$ — no scan (the upper bound is the same routine with $>$ in place of $\geq$). The one line that matters is `hi = mid` rather than `hi = mid - 1`: keeping `mid` as a candidate is precisely what pinpoints the *first* match instead of discarding it.

**Board: trace `lowerBound` (instance 1).** On the sorted array `[1, 3, 3, 3, 5, 7]`, find the first index with `A[i] >= 3`. Write `lo = 0, hi = 6`. Iteration 1: `mid = 3`, `A[3] = 3` which is *not* $< 3$, so `hi = mid = 3`. Iteration 2: `lo = 0, hi = 3, mid = 1`, `A[1] = 3` not $< 3$, so `hi = 1`. Iteration 3: `lo = 0, hi = 1, mid = 0`, `A[0] = 1 < 3`, so `lo = 1`. Now `lo == hi == 1`; return 1 — the first 3. Build order: bounds, then each iteration's `mid` and the surviving half-open range, underlining the `hi = mid` line each time it fires. **Live question:** "when `A[mid]` *equals* the key, why do we not return mid?" (Because an earlier equal key may exist to the left; keeping `mid` as a candidate and continuing left finds the first one.)

**Board: distinct instance 2 — count the duplicates.** Use the sorted `[1, 2, 2, 2, 2, 3]`. Send one student to trace `lowerBound(A, 6, 2)` and another to trace the upper bound ($>$ variant). Lower bound lands on index 1 (first 2); upper bound lands on index 5 (one past the last 2). The occurrence count is $5 - 1 = 4$. **Prediction hook before the second trace:** "how many 2s are there, and can we get that without ever scanning the array?" (Four, and yes — two $O(\log n)$ searches, no scan.)

> **Teaching note:** The frequent slip is writing `hi = mid - 1` out of habit and then hunting for the bug when duplicates are miscounted. Anchor it by contrast, on the board: in the plain search, an equal element at `mid` is a reason to *stop*; in the lower-bound search, it is a reason to keep `mid` as the best-so-far *and keep looking left*. The two routines look nearly identical and mean opposite things at exactly this line. Also note the loop guard differs — `while (lo < hi)` here, `while (lo <= hi)` in the plain search — because the ranges are half-open versus inclusive; mixing them up is the second-most-common bug in this section.

| algorithm | precondition | best | worst | space |
|---|---|---|---|---|
| linear search | none | $O(1)$ | $\Theta(n)$ | $O(1)$ |
| binary search (iterative) | sorted | $O(1)$ | $\Theta(\log n)$ | $O(1)$ |

Binary search's $\Theta(\log n)$ worst case looks strictly better — and it is, *once you own sorted data*. The catch is the precondition: it costs something to establish, and it must be maintained after every insertion. The rest of the lecture is about that cost — and before we leave search, two problems where binary search finds an answer that is *not* stored in any array at all.

Concepts That Click

## 8.4 Two Applications of Binary Search

Binary search is not only for arrays of stored data. Whenever a yes/no question about a value has a single true→false flip point (a **monotone predicate**), binary search finds that point in $O(\log(\text{range}))$ probes — even when no element of an array is involved. Two applications make the pattern concrete.

### 8.4.1 Application 1: the integer square root of $n$

Suppose we want $\lfloor\sqrt{n}\rfloor$ — the largest integer $k$ with $k^2 \le n$ — using only integer arithmetic, no floating point. The answer $k$ is not stored in any array. The trick is to search the *answer space* $0..n$ instead of an array: define the predicate $P(k) = (k^2 \le n)$. As $k$ grows, $P$ is *true* up to a point and *false* after it — a single true→false boundary. A monotone predicate is exactly what binary search needs: find the largest $k$ with $P(k)$ true, and that $k$ is $\lfloor\sqrt{n}\rfloor$.

The deck's code (standard treatment; Karumanchi Ch. 11, pp.547–589 (PDF)):

```
int isqrt(int n) {                        // floor(sqrt(n)), n >= 0
  int lo = 0, hi = n, ans = 0;
  while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if ((long long) mid * mid <= n) {     // P(mid) true
      ans = mid; lo = mid + 1;            // mid is feasible; push right
    } else hi = mid - 1;                  // P(mid) false; push left
  }
  return ans;                             // largest feasible k
}
```

The `(long long)` cast widens *before* the multiply: `mid` can reach $n$, and $n^2$ overflows `int` — and even a 32-bit `long` on Windows (LLP64). The cost is $O(\log n)$ probes and $O(1)$ space: the range $0..n$ halves each step, no array, no floating point.

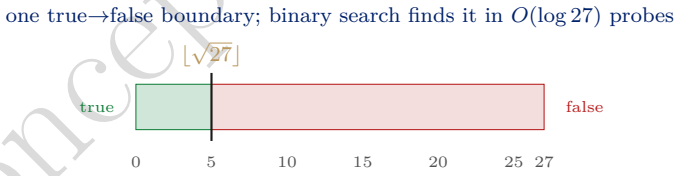



Figure 8.3: The square-root search space: $P(k) = (k^2 \le 27)$ is true for $k \le 5$ and false after, and binary search finds the single boundary (cf. Karumanchi, Ch. 11, pp.547–589 (PDF)).

**Board: the predicate table and boundary (instance 1).** Draw the little table of $k$, $k^2$, and $P(k)$ for $k = 0..7$ first, then draw the two-colour band underneath with the boundary rule at $k = 5$. Build order: table first (so students see $k^2$ cross 27 between 5 and 6), then the band, then the rule. **Live question:** "we never stored the number 5 anywhere — where did the answer come from?" (From the boundary of a predicate: $P(5)$ is the last true value, $P(6)$ the first false, so $\lfloor\sqrt{27}\rfloor = 5$, found in $O(\log 27)$ probes.)

**Board: distinct instance 2 — $\lfloor\sqrt{50}\rfloor$, and the overflow.** A genuinely different instance: compute $\lfloor\sqrt{50}\rfloor$ by the same algorithm. Boundary reasoning first: $7^2 = 49 \le 50$, $8^2 = 64 > 50$, so the answer is 7. Then trace the actual probes: `lo=0, hi=50, ans=0, mid=25` ($625 > 50 \to$ `hi` $= 24$); `mid=12` ($144 > 50 \to$ `hi` $= 11$); `mid=5` ($25 \le 50 \to$ `ans` $= 5$, `lo` $= 6$); `mid=8` ($64 > 50 \to$ `hi` $= 7$); `mid=6` ($36 \le 50 \to$ `ans` $= 6$, `lo` $= 7$); `mid=7` ($49 \le 50 \to$ `ans` $= 7$, `lo` $= 8$); `lo>hi`, return 7. Now the

Concepts That Click

overflow demonstration: write `mid = 1,000,000` and compute `mid * mid = 1,000,000,000,000` — far past `int`'s range; the `(long long)` cast is what keeps the product correct. **The live hook:** pose the range-of-probes question before tracing — "about how many probes for $n = 50$?" ($\lceil \log_2 51 \rceil \approx 6$ — the trace confirms it.)

> **Teaching note:** The conceptual leap is "binary search over a space where nothing is stored." Students who have only seen binary search in arrays have trouble finding the "middle" of an answer space. Give them the sentence explicitly: the array's indices $0..n-1$ have become the candidate answers $0..n$, and the "sorted order" that was assumed of the data is now a property of the *predicate* — it goes true, true, …, true, false, false, …once and never flips back. Also state plainly that the `(long long)` is not decoration: without it the widening does not happen before the multiply and a large `n` silently overflows.

### 8.4.2 Application 2: the longest common prefix

The calculator's autocomplete must find the longest prefix shared by all identifiers starting with a typed stem. For "flower", "flow", "flight" the shared prefix is "fl". Two facts make this a binary-search problem:

1. If the strings are *sorted*, the longest common prefix of the whole set equals the common prefix of the first and the last string — so only two strings need comparing.
2. The length $L$ of the common prefix of two strings is itself found by binary search: the predicate "the first $L$ characters are equal" is true for small $L$ and false beyond the answer — again a single true→false boundary.

So: sort the set once ($O(n \log n \cdot m)$ for strings of length up to $m$), then answer each prefix query in $O(m)$ using the first and last strings — no scan over all $n$ strings. The deck's code (standard treatment; Karumanchi Ch. 15, pp.640–684 (PDF), string search):

```
int lcp2(const char *s, const char *t) {   // common prefix of s, t
  int lo = 0, hi = min(strlen(s), strlen(t)), ans = 0;
  while (lo <= hi) {
    int L = lo + (hi - lo) / 2;
    if (prefixEq(s, t, L)) { ans = L; lo = L + 1; }  // P(L) true
    else hi = L - 1;                                  // P(L) false
  }
  return ans;
}
// For a SORTED set: LCP of all = lcp2(first, last).
```

`prefixEq(s, t, L)` costs $O(L)$; trying $O(\log m)$ candidate lengths makes the query $O(m \log m)$ — or $O(m)$ if each check is made incremental.

**Board: draw the sorted strings and the shared prefix (instance 1).** Draw the three rows as letter cells in sorted order (`flight`, `flow`, `flower`), top to bottom. Then compare column by column, colouring each column that agrees across the first and last rows: column 0 ("f") agrees, column 1 ("l") agrees, column 2 ("i" vs "o") is the first disagreement — stop and draw the brace around the first two columns, labelled "fl". Build order: three sorted rows, then the column-by-column comparison of first vs. last, then the brace. Say the key fact as you stop: you never compared the middle row at all.

**Board: distinct instance 2 — a different set, and the prediction.** Give a fresh set `["apple", "apply", "apt"]` (already sorted). Compare the first and last strings `"apple"` and

Concepts That Click

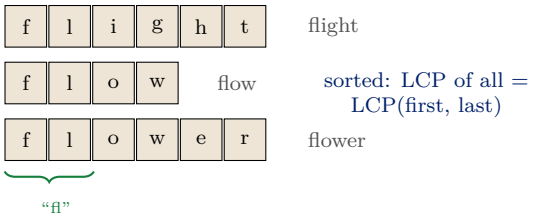



Figure 8.4: The longest common prefix of a sorted set equals the LCP of its first and last strings: "fl" here (cf. Karumanchi, Ch. 15, pp.640–684 (PDF)).

`"apt"`: position 0 is "a" in both, position 1 is "p" in both, position 2 is "p" vs "t" — the first disagreement. The shared prefix is "ap", length 2. **The live hook is the deck's prediction:** for the original set `["flight", "flow", "flower"]`, "all share a prefix of length $L$" — for which $L$ is it first *false*? (A) $L = 1$, (B) $L = 2$, (C) $L = 3$. Let the class vote, then resolve: $L = 1$ ("f") is true; $L = 2$ ("fl") is still true, so the boundary lies *after* this; $L = 3$ would require "fli" and "flo" to agree, and "flight" has an "i" where "flow" has an "o" — first false at $L = 3$. Answer **C**; the longest common length is 2, the largest $L$ still true.

> **Teaching note:** Students resist the sorted-set fact: "surely the middle string could break the consensus?" It cannot, and the reason is the whole value of sorting — once sorted, the first and last strings are the two extremes, and every string between them shares at least their common prefix. Make the class *try* to construct a counterexample (a middle string that shares less than "fl") and watch them fail; that failure is the proof, delivered by the class, which is worth more than a sentence from you. Note also the honest scope: this query is $O(m)$ or $O(m \log m)$ in the string length, not $O(\log n)$ in the set size — sorting the set is what removed the $O(n)$ scan, and binary search on the *length* is a separate, inner use of the same idea.

## 8.5 Creating Order: The Four Elementary Sorts

Binary search is the reward; sorting is the price we pay for it. If the identifier table is sorted once and read many times, that price is amortized away. But sorting is not only for search: it puts "equal" things next to each other, which is how we deduplicate, find duplicate entries, group records, and compute medians. The calculator's symbol table, its autocomplete set, and its report of repeated declarations all begin with a sort.

Four algorithms are introduced here — not as a list to memorize but as four answers to the question "what is the cheapest way, given *this* input?":

| sort | idea | signature |
|---|---|---|
| insertion | keep $A[0..i-1]$ sorted; insert $A[i]$ into place | adaptive: near-sorted input is fast |
| selection | find the minimum of the unsorted part; swap it forward | exactly $n-1$ swaps, always $\Theta(n^2)$ comparisons |
| bubble | swap adjacent inversions; the largest "bubbles" to the end | can stop early when a pass makes no swap |

All three sorts above sort *by comparing pairs* of elements, so they work on any comparable type. Counting sort, introduced later in this section, breaks that property and gets linear time — but only for keys drawn from a small integer range. Two pieces of vocabulary decide many real choices:

Concepts That Click

**Definition (Stable sort).** *A sort is **stable** if equal keys keep their original relative order.*

**Definition (In-place sort).** *A sort is **in-place** if it uses $O(1)$ auxiliary space beyond the input.*

Reference treatment: Knuth, *TAOCP* Vol. 3; Horowitz & Sahni Ch. 7 — both chapter-level, not page-indexed.

### 8.5.1 Insertion sort: grow a sorted prefix

The deck's insertion sort, written in C (Wirth §2.2.1, p.46; Karumanchi Ch. 10, pp.504–546 (PDF)):

```
void insertionSort(int A[], int n) {
  for (int i = 1; i < n; i++) {
    int key = A[i], j = i - 1;          // key is the element to place
    while (j >= 0 && A[j] > key) {      // shift larger elements right
      A[j + 1] = A[j];
      j--;
    }
    A[j + 1] = key;                     // drop key into its slot
  }
}
```

The invariant is: before iteration $i$, the subarray $A[0..i-1]$ is sorted. The outer loop extends that sorted prefix by one element; the inner loop shifts the larger elements rightward until `key` finds its place. When the prefix is already sorted, the inner loop runs *once* — which is the whole point of an adaptive sort.

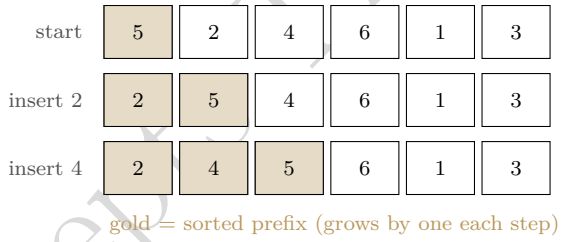



Figure 8.5: Insertion sort of `[5, 2, 4, 6, 1, 3]`: the gold sorted prefix grows by one each step (cf. Wirth §2.2.1, p.46).

**Board: the full trace, step by step (instance 1).** Start with the prefix `[5]`. Then insert each remaining element, narrating the shifts: insert 2 (shift 5) ⇒ `[2, 5, 4, 6, 1, 3]`; insert 4 (shift 5) ⇒ `[2, 4, 5, 6, 1, 3]`; insert 6 (no shift, $6 > 5$) ⇒ unchanged; insert 1 (shift $6, 5, 4, 2$) ⇒ `[1, 2, 4, 5, 6, 3]`; insert 3 (shift $6, 5, 4$) ⇒ `[1, 2, 3, 4, 5, 6]`. Build order: write the array once, then re-draw the gold prefix growing by one cell per row. Underline step 3 — it is the adaptive case in miniature: the prefix was already sorted, so the inner loop stopped after a single comparison and no shift.

**Board: distinct instance 2 — `[4, 2, 5, 1, 3]`.** A different input, to be traced by the class rather than by you. After $i = 1$ (key 2): `[2, 4, 5, 1, 3]` (one shift). After $i = 2$ (key 5): unchanged, no shift ($5 > 4$). After $i = 3$ (key 1): shift $5, 4, 2$ ⇒ `[1, 2, 4, 5, 3]`. After $i = 4$ (key 3): shift $5, 4$ ⇒ `[1, 2, 3, 4, 5]`. Total shifts: $1 + 0 + 3 + 2 = 6$. **Live hook:** pose the already-sorted input — "how many shifts does `[1, 2, 3, 4, 5]` cost?" (Zero shifts; $n - 1$ comparisons; the inner loop stops immediately every time. That is the $O(n)$ best case.)

Concepts That Click

### 8.5.2 Insertion sort: the bill

Count the shifts (the inner-loop body). The outer loop runs $i = 1 \ldots n-1$, so there are $n-1$ insertions:

| case | comparisons / shifts | example input |
|---|---|---|
| best | $n-1$ (each key already in place) | already sorted: $O(n)$ |
| worst | $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$ | reverse sorted: $\Theta(n^2)$ |
| average | $\Theta(n^2)$ | random order |

Insertion sort is adaptive: its cost falls to $O(n)$ on nearly-sorted input — it is the only one of the three comparison sorts with a linear best case. It is also stable (equal keys stay in order, because we shift only on `A[j] > key`, not $\geq$) and in-place ($O(1)$ extra space). Those last two properties are why insertion sort is the base case inside efficient sorts in Week 9.

**Board: derive the worst-case sum.** Do not just quote $\frac{n(n-1)}{2}$ — build it: on reverse-sorted input the $i$-th key must shift past all $i$ elements before it, so the total is $1+2+\cdots+(n-1)$. Write the sum, then the closed form. **Board: the stability trace (second board moment).** Take `[(A,2), (B,2)]` (two equal keys, A before B) and trace one insertion: the inner loop shifts only while `A[j] > key`, and since $2 \not> 2$ it stops — A stays before B. **The live hook:** "what single character in the code guarantees stability?" (The strict `>`; had it been `>=`, equal keys would be shifted and the order would swap.)

### 8.5.3 Selection sort: repeatedly pick the minimum

The deck's selection sort, written in C (Wirth §2.2.2, p.48; Karumanchi Ch. 10, pp.504–546 (PDF)):

```
void selectionSort(int A[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int m = i;                          // index of the minimum so far
    for (int j = i + 1; j < n; j++)
      if (A[j] < A[m]) m = j;           // find the smallest in A[i..n-1]
    swap(&A[i], &A[m]);                 // put it at the front of the rest
  }
}
```

The invariant is: after iteration $i$, the subarray $A[0..i]$ holds the $i+1$ smallest elements, in final position. The scan finds the next minimum, and one swap moves it into place. Note that the scan always covers the whole unsorted remainder — regardless of whether the input is already sorted.

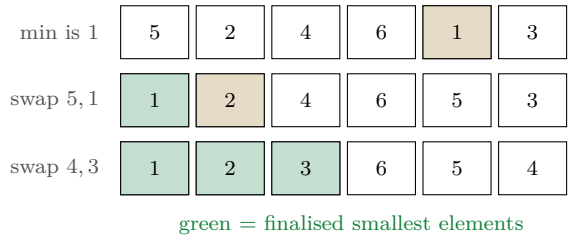



Figure 8.6: Selection sort of `[5, 2, 4, 6, 1, 3]`: green marks finalised smallest elements, gold the current minimum (cf. Wirth §2.2.2, p.48).

**Board: the scan-and-swap trace (instance 1).** Row 1: scan the whole array, find 1 at index 4 (mark it gold), swap it with index 0 (value 5) $\Rightarrow$ `[1, 2, 4, 6, 5, 3]`, index 0 final

Concepts That Click

(green). Row 2: scan `A[1..5]`, find 2 (already at index 1), no-op swap. Row 3: scan `A[2..5]`, find 3 at index 5, swap with index 2 (value 4) ⇒ `[1, 2, 3, 6, 5, 4]`, indices 0..2 final. Build order: current array, then gold the minimum found, then green the newly finalised slot. Underline that the scan *always* covers the whole remainder.

**Board: distinct instance 2 — the instability trace.** Take `[2_a, 2_b, 1]` (label the two 2s so their order is visible). The scan finds the minimum 1 at index 2; the long-range swap puts it at index 0, giving `[1, 2_b, 2_a]` — the two equal keys have *changed order*. **Prediction hook:** before the swap, ask "are these two 2s equal? then can selection sort ever move one past the other?" (Yes — the swap leaps index 0 over index 2, dragging `2_a` past `2_b`. That is exactly why selection sort is not stable.)

### 8.5.4 Selection sort: the bill (and a warning)

The inner-loop comparisons do not depend on the data at all: $\sum_{i=0}^{n-2}(n-1-i) = \dfrac{n(n-1)}{2}$ — every input.

| case | comparisons | swaps |
|---|---|---|
| best | $\Theta(n^2)$ | 0 (already sorted) |
| worst | $\Theta(n^2)$ | $n-1$ |

Selection sort is not adaptive: a sorted input is scanned just as hard as a reversed one, always $\Theta(n^2)$ comparisons. Its one virtue is the minimum number of *writes* ($n-1$ swaps), useful when moving data is far costlier than comparing it. It is not stable: the long-range swap can leap an equal key over its twin. It is, however, in-place: $O(1)$ auxiliary space.

**Board: count the comparisons on sorted input.** Ask the class to count comparisons for `[1, 2, 3, 4, 5]`: the outer loop still scans $4+3+2+1 = 10$ times even though no swap happens. **Live question:** "when is selection sort the right tool despite being $\Theta(n^2)$?" (When moving data is very expensive — exactly $n-1$ swaps is the minimum possible number of writes for a sort.)

### 8.5.5 Bubble sort: swap neighbours, bubble the max up

The deck's bubble sort with an early exit, written in C (Wirth §2.2.3, p.50; Karumanchi Ch. 10, pp.504–546 (PDF)):

```
void bubbleSort(int A[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int swapped = 0;
    for (int j = 0; j < n - 1 - i; j++)
      if (A[j] > A[j + 1]) { swap(&A[j], &A[j + 1]); swapped = 1; }
    if (!swapped) break;     // a clean pass: already sorted
  }
}
```

The invariant is: after pass $i$, the largest $i+1$ elements are final at the right end, so each pass can stop $i$ slots earlier. The `swapped` flag is the practical touch: a pass with no swap proves the array is sorted, so we stop.

**Board: one full pass (instance 1).** Compare neighbours and swap when out of order: $(5,2)$ swap ⇒ `[2, 5, 4, 6, 1, 3]`; $(5,4)$ swap ⇒ `[2, 4, 5, 6, 1, 3]`; $(5,6)$ no swap; $(6,1)$ swap ⇒ `[2, 4, 5, 1, 6, 3]`; $(6,3)$ swap ⇒ `[2, 4, 5, 1, 3, 6]`. After one pass the maximum 6 has bubbled to the end and is final (mark it gold). Build order: the starting row, then the pass drawn

Concepts That Click

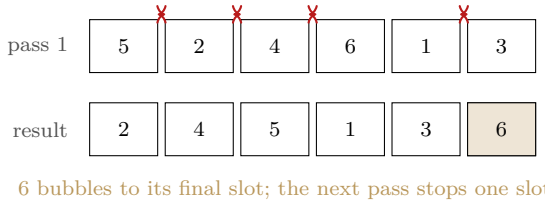



Figure 8.7: One bubble pass over `[5, 2, 4, 6, 1, 3]`: adjacent swaps push the maximum 6 to its final slot (cf. Wirth §2.2.3, p.50).

as the value moving rightward one comparison at a time, then the finalised gold cell. Point out the next pass stops one slot sooner.

**Board: distinct instance 2 — the early exit.** Trace `[1, 2, 4, 3]`: $(1,2)$ no swap, $(2,4)$ no swap, $(4,3)$ swap $\Rightarrow$ `[1, 2, 3, 4]`. That pass *did* swap, so a second pass runs: $(1,2)$, $(2,3)$, $(3,4)$ — no swaps, so `swapped` stays 0 and the loop breaks. Two passes total. **Live hook:** "on already-sorted input `[1, 2, 3, 4, 5]`, how many passes does the *plain* bubble sort make, and how many does this flagged version make?" (Plain: $n-1$ passes, all doing nothing, $\Theta(n^2)$. Flagged: one pass, then break — $O(n)$. The flag is the difference between adaptive and not.)

### 8.5.6 Bubble sort: the bill

| case | comparisons | note |
|---|---|---|
| best | $n-1$ | sorted input: pass 1 swaps nothing, early exit |
| worst | $\frac{n(n-1)}{2}$ | reverse sorted: $\Theta(n^2)$ |
| average | $\Theta(n^2)$ | random order |

Bubble sort is adaptive only *with* the early-exit flag; the naive version is $\Theta(n^2)$ on every input, including sorted data. It is stable (equal adjacent keys are never swapped, because we swap only on strict $>$ — their relative order is preserved) and in-place. Its comparison count equals insertion sort's on average, but bubble sort does far more *swaps*; insertion sort is the better practical choice among the three. All three comparison sorts hit the same $\Theta(n^2)$ wall. Week 9 asks whether any comparison sort can do better — and counting sort, next, sidesteps comparisons entirely.

**Board: the swap-count comparison (hook).** On the *same* reverse-sorted input, insertion sort and bubble sort make the same number of comparisons but insertion makes roughly *half* as many element moves (each shift moves an element one slot; a bubble swap moves two). Write the two counts side by side and ask: "same comparisons — so why is insertion sort the practical choice?" (Fewer data movements; on a machine where a move costs the same as a comparison, that halves the constant.)

### 8.5.7 Counting sort: sort without comparing

All three sorts above compare keys pairwise. But if the keys are small non-negative integers, the key *is already an array index* — so it can be counted, not compared. Counting sort tallies how many times each value occurs, converts those counts to positions with a running total, and scatters each element to its slot. It never asks "is $A[i] < A[j]$?" The precondition is that keys lie in a known small range $0..k-1$. The trade is linear time for $\Theta(n+k)$ extra space — and it is only correct for keys that fit that range. In the calculator, counting the frequency of each byte or token class, and bucketing identifiers by length, are both one-pass counting sorts — no comparisons at

Concepts That Click

all. The deck's code (Karumanchi Ch. 10, pp.504–546 (PDF); Horowitz & Sahni Ch. 7, standard treatment):

```
void countingSort(int A[], int n, int k) {   // keys in 0..k-1
  int *count = calloc(k, sizeof(int));       // zero-initialised tally
  for (int i = 0; i < n; i++) count[A[i]]++;
  for (int v = 1; v < k; v++) count[v] += count[v - 1];  // prefix sums
  int *out = malloc(n * sizeof(int));
  for (int i = n - 1; i >= 0; i--)           // reverse pass => stable
    out[--count[A[i]]] = A[i];
  for (int i = 0; i < n; i++) A[i] = out[i];
  free(out); free(count);                    // Week 1: every malloc freed
}
```

`calloc(k, ...)` zeroes the tally, so no counting slot is read before it is written (Week 1's `calloc` guarantee, reused). The reverse pass over $A$ makes equal keys land in their original relative order — that is what makes this sort stable.

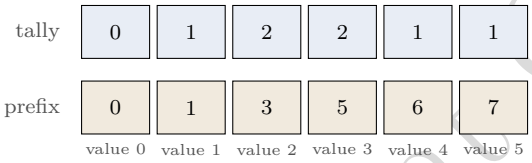



Figure 8.8: Counting sort of $A = [4, 2, 2, 5, 3, 3, 1]$: the tally row counts each value, the prefix row gives the scatter positions (cf. Karumanchi, Ch. 10, pp.504–546 (PDF)).

**Board: tally, then prefix, then scatter (instance 1).** For $A = [4, 2, 2, 5, 3, 3, 1]$, keys in 0..5 so $k = 6$. Tally row: value 0 occurs 0 times, 1 once, 2 twice, 3 twice, 4 once, 5 once ⇒ `[0, 1, 2, 2, 1, 1]`. Prefix row (running total): `[0, 1, 3, 5, 6, 7]`. Build order: write the labels "value 0..5" first, then the tally row, then the prefix row as a running sum (each entry equals the previous plus the tally above it). Narrate the meaning: prefix[v] is where the next occurrence of value $v$ goes during the scatter. Then scatter *in reverse* ($i$ from $n-1$ down to 0), placing each element into `--count[A[i]]`, and underline that the reverse pass is what preserves equal keys' order.

**Board: distinct instance 2 — `[2, 0, 3, 2, 1, 3, 0]`.** A fresh input, $k = 4$. Tally: value 0 twice, 1 once, 2 twice, 3 twice ⇒ `[2, 1, 2, 2]`. Prefix: `[2, 3, 5, 7]`. Scatter in reverse and the sorted array is `[0, 0, 1, 2, 2, 3, 3]`. **Prediction hook before the scatter:** "given the prefix row, where does the last 0 in the input (value 0) land, and where does the 1 land?" (The 1 has prefix index 3, so it goes to slot 3; the two 0s fill slots 1 then 0 because the scatter is right-to-left — stability in action.)

> **Teaching note:** The range precondition is the trap. Students see "count occurrences" and reach for counting sort everywhere; the honest bill is $\Theta(n+k)$ *space*, and if the keys span $0..10^9$ the tally array is enormous — counting sort is then worse than useless. The second trap is stability: it is the *reverse* scatter pass that makes the sort stable, not the tally itself; a forward scatter would reverse equal keys' order. Anchor both: "pay for the range; scatter backwards."

Concepts That Click

### 8.5.8 Counting sort: the bill

| phase | work | cost |
|---|---|---|
| tally | one pass over $A$ | $\Theta(n)$ |
| prefix sums | one pass over $0..k-1$ | $\Theta(k)$ |
| scatter | one reverse pass over $A$ | $\Theta(n)$ |
| total | | $\Theta(n+k)$ time, $\Theta(n+k)$ space |

Counting sort is linear time when $k = O(n)$. But watch the range: if keys span a huge range, the tally array is enormous and counting sort is worse than useless. It is stable (with the reverse scatter) but not in-place ($\Theta(n+k)$ auxiliary). Crucially, it sorts *without comparing keys* — so the $\Theta(n \log n)$ comparison lower bound of Week 9 does not apply to it.

### 8.5.9 The four sorts, compared

| sort | best | worst | space | stable | in-place |
|---|---|---|---|---|---|
| insertion | $O(n)$ | $\Theta(n^2)$ | $O(1)$ | yes | yes |
| selection | $\Theta(n^2)$ | $\Theta(n^2)$ | $O(1)$ | no | yes |
| bubble | $O(n)$ | $\Theta(n^2)$ | $O(1)$ | yes | yes |
| counting | $\Theta(n+k)$ | $\Theta(n+k)$ | $O(n+k)$ | yes | no |

Read the table as a workload guide, not a ranking: **nearly sorted** input → insertion sort, whose $O(n)$ best case is the whole point; **small integer keys** in a known range → counting sort, linear but at the cost of $O(n+k)$ space; and when **minimal writes** matters more than comparisons → selection sort, with its exactly $n-1$ swaps. No single winner — every row is right for *some* input. The skill is stating the workload, then choosing.

**Board: reconstruct the table row by row (instance 1).** Do not copy the table — build it. Write the four sort names down the left, then fill best/worst from the derivations the class just did (insertion: $O(n)$ best because of the strict-> stop; selection: $\Theta(n^2)$ always; bubble: $O(n)$ best only with the flag; counting: $\Theta(n+k)$ because the key is an index). Add the stability and in-place columns last, each with a one-word justification. **Live-interaction hook:** pose the two workloads before revealing — "(i) an already almost-sorted array; (ii) $10^6$ ages in 0..120 — which elementary sort fits each best?" (i) insertion sort — nearly every key is already in place, so the run is $O(n)$; (ii) counting sort — $k = 121 \ll n$, so $\Theta(n+k) \approx \Theta(n)$. Selection or bubble would be $\Theta(n^2)$ on both — the wrong tool for either input.

**Board: distinct instance 2 — two more loads, one straight from the table.** Give a fresh pair and have the class place each on the table before you confirm: "(iii) a reverse-sorted array of $10^5$ keys, and (iv) $10^6$ keys that are already sorted." (iii) every comparison sort is worst-case $\Theta(n^2)$, but selection sort makes exactly $n-1$ swaps versus insertion sort's $\Theta(n^2)$ shifts — if writes are expensive, selection wins on moves; (iv) insertion and bubble both drop to $O(n)$ (bubble only with the flag), while selection stays $\Theta(n^2)$ and counting pays its $\Theta(n+k)$ space. Land the reading of the table: "best/worst" answers "what shape of input," and stable/in-place answers "what does the job cost" — together they pick the tool.

## 8.6 Stability, In-Place, and the Classic Mistakes

**Stable** means equal keys keep their original relative order. This matters when records are sorted by a *secondary* key after already being sorted by a primary one — a stable sort preserves the earlier ordering within ties.

Concepts That Click

**Example 8.6.1** (Worked example — stability). *Sort* `[(A,2), (B,1), (C,2)]` *by the number. A stable sort yields* `[(B,1), (A,2), (C,2)]` *—* `A` *stays before* `C`*, since both carry key* 2 *and* `A` *came first. An unstable sort may return* `[(B,1), (C,2), (A,2)]` *— same keys, different record order.*

**Board: the multi-key sort (instance 1).** Show the record-sort scenario concretely: a class list sorted by *name* first, then re-sorted by *grade*; a stable sort keeps students with the same grade in name order, an unstable one scrambles them. Draw the before-list, then the two possible after-lists side by side. Build order: records, primary sort, then the secondary sort under both stability regimes. **Board: distinct instance 2 — labeled equal keys.** On `[3_a, 1, 3_b, 2]`, ask which of insertion, selection, bubble, and counting are stable and what tie order a stable sort leaves: insertion, bubble, and counting are stable; selection is not; a stable result is `[1, 2, 3_a, 3_b]`. **Live hook:** "point to the one code line that makes selection sort unstable" (the long-range `swap(&A[i], &A[m])` — it can leap an equal key over its twin).

**In-place** means $O(1)$ auxiliary space beyond the input. Insertion, selection, and bubble sort are in-place; counting sort needs a $\Theta(n+k)$ output buffer and a $\Theta(k)$ tally, so it is not. Summing the properties: stable sorts here are insertion, bubble, and counting; not stable is selection, whose long-range swap can leap an equal key.

> **Teaching note:** The stability work only lands if students can see *why* it matters, and the multi-key sort is the reason: "sort by name, then by grade" must not destroy the name order within a grade. Make the point with the labeled-equal-keys trace, then tie it forward — stability is not academic trivia; it is the property radix sort (Week 9) is entirely built on, which is why it is introduced here.

### 8.6.1 Three classic mistakes

Binary searching unsorted data — no crash, just a plausible wrong answer. The precondition is a contract: sort first, or use linear search. Forgetting the overflow in `mid` — `(lo + hi)/2` overflows `int` for large indices; write `lo + (hi - lo)/2`. The same trap appears as `mid * mid` in `isqrt` and the `(long long)` casts in C. Claiming a sort is $\Theta(n \log n)$ "because it sorts" — the elementary sorts are all $\Theta(n^2)$ worst case. State the *case* and the *bound*, and check whether the algorithm even compares keys (counting sort does not).

How to revise: for each sort, write one sentence naming the invariant, the best case, the worst case, and whether it is stable — then trace one pass by hand. The test is traces and cases, not definitions.

> **Teaching note:** These three mistakes are diagnostic, not decorative — each maps to a specific conceptual hole. The first says the student treats the precondition as a hint; the second says they ignore C's fixed-width arithmetic; the third says they conflate a *lower bound on a class of algorithms* with the cost of any particular algorithm. If a student makes the third mistake, the fix is not "memorize the table" but "state the case, state the bound, and ask whether the algorithm compares keys at all."

## 8.7 Summary

**The one-sentence spine:** order turns an $O(n)$ scan into an $O(\log n)$ probe, and the elementary sorts are the cheapest ways to buy that order for a given input — each with a different bill.

Concepts That Click

**The search contract**: linear search needs no order, $\Theta(n)$ worst case; binary search *requires* sorted input and is $\Theta(\log n)$ worst case — but is silently wrong on unsorted data. The lower/upper-bound variants return the first/last occurrence in $O(\log n)$, and their difference counts duplicates without a scan.

**Binary search on a monotone predicate** answers questions about values that are not stored: $\lfloor\sqrt{n}\rfloor$ (search the answer space $0..n$; widen `mid*mid` with `(long long)`), and the longest common prefix (search the prefix length; sort once, then compare only the first and last strings).

**Elementary sorts**, with their honest bills:

| sort | best | worst | space | stable | in-place |
|---|---|---|---|---|---|
| insertion | $O(n)$ | $\Theta(n^2)$ | $O(1)$ | yes | yes |
| selection | $\Theta(n^2)$ | $\Theta(n^2)$ | $O(1)$ | no | yes |
| bubble | $O(n)$ | $\Theta(n^2)$ | $O(1)$ | yes | yes |
| counting | $\Theta(n+k)$ | $\Theta(n+k)$ | $O(n+k)$ | yes | no |

**Dictionary at the end**: *stable* = equal keys keep their order (insertion, bubble, counting yes; selection no); *in-place* = $O(1)$ auxiliary (all but counting); the three properties — stable, in-place, best/worst case — decide which sort fits an input. Every elementary sort is $\Theta(n^2)$ worst case; counting sort is the only one that escapes comparisons, and pays $O(n+k)$ space for it.

**Bridge to Week 9.** All three comparison sorts hit the same $\Theta(n^2)$ wall, and counting sort sidestepped comparisons entirely. Week 9 asks whether any *comparison* sort can beat $\Theta(n \log n)$ — and which algorithms escape that bound: quick, merge, shell, radix, heap.

Concepts That Click